Módulo 1: Git Local, Configuración y Conexión Remota

Guía rápida de preparación para examen sobre fundamentos de control de versiones con Git.

1. Inicialización y Configuración Inicial (`git init`)

Propósito: Transformar una carpeta normal del sistema de archivos en un repositorio de Git para comenzar a registrar el historial de cambios.

Secuencia de Comandos

1. **Configurar la identidad del usuario** (Obligatorio la primera vez en el equipo):
   ```
   git config --global user.name "Tu Nombre"
   git config --global user.email "tu@email.com"
   ```
2. **Navegar al directorio del proyecto:**
   ```
   cd /ruta/de/tu/carpeta
   ```
3. **Inicializar el repositorio:**
   ```
   git init
   ```

2. Exclusión de Archivos (`.gitignore`)

Propósito: Prevenir que Git realice el seguimiento o suba al remoto archivos temporales, credenciales de acceso, claves privadas o directorios pesados de dependencias.

Implementación

Cree un archivo nombrado exactamente `.gitignore` en la raíz del proyecto.

Ejemplo de Configuración

Regla / Patrón	Descripción del Comportamiento
`node_modules/`	Ignora el directorio completo de dependencias de Node.js.

Regla / Patrón	Descripción del Comportamiento
`*.log`	Ignora cualquier archivo que termine con la extensión `.log`.
`.env`	Ignora el archivo de variables de entorno con credenciales sensibles.

3. Estados y Commits (`git add` y `git commit`)

Flujo de trabajo básico: `Working Directory` $\rightarrow$ `Staging Area` $\rightarrow$ `Repository (Commit)`

A) Preparación de Cambios (`git add`)

- `git add archivo.html` — Añade un archivo específico al Staging Area.
- `git add .` — Añade todos los archivos modificados, eliminados o nuevos del directorio actual al Staging Area.

B) Guardado de Versiones (`git commit`)

Opción Estándar (`-m`)

Requiere la ejecución previa del comando `git add`.

```
git add .
git commit -m "Añadir estructura inicial de la web"
```

Atajo Directo (`-am`)

Combina la fase de adición y la confirmación en un solo paso únicamente para archivos que ya están bajo el seguimiento de Git (*tracked*).

```
git commit -am "Cambiar el título principal"
```

Atención para el Examen: El parámetro `-am` solo procesa archivos modificados previamente rastreados. Si se crea un archivo nuevo (*untracked*), `-am` **no** lo incluirá. Para archivos nuevos es obligatorio ejecutar git add en primer lugar.

4. Gestión de Crisis: `git reset` vs `git revert`

Comando	Tipo de Operación	Ámbito de Uso	Efecto en el Historial
`git reset`	Destructiva / Modificadora	**Solo Local** (antes de hacer `push`)	Reescribe e elimina commits del historial.
`git revert`	Conservadora / Segura	**Remoto / Público** (después de hacer `push`)	Agrega un nuevo commit que deshace los cambios.

Estrategia con `git reset`

Uso exclusivo en entorno local. Si se ejecuta tras un `push`, romperá la sincronización con el repositorio remoto.

- **Soft Reset:** Conserva las modificaciones en el área de trabajo.
  `git reset --soft HEAD^` — Deshace el último commit, pero mantiene las ediciones intactas en el código para permitir corregir el mensaje o ajustar los archivos.
- **Hard Reset:** Eliminación definitiva de cambios.
  `git reset --hard HEAD^` — Deshace el último commit y descarta totalmente los cambios en los archivos, regresando al estado exacto del commit anterior.

Estrategia con `git revert`

Obligatorio cuando los cambios ya han sido enviados a GitHub. No altera el historial existente.

1. Identificar el identificador del commit a revertir:
   `git log --oneline`
2. Aplicar la reversión mediante el hash:
   `git revert HASH_DEL_COMMIT`

5. Conexión Remota y Subida a la Nube (`git remote` + `git push`)

Establece el enlace de comunicación entre el repositorio local y la plataforma remota (GitHub). Se configura una sola vez por proyecto.

Pasos de Ejecución

1. **Vincular el repositorio local con la URL remota:**
   `git remote add origin URL_DE_TU_REPOSITORIO`
2. **Verificar la configuración de enlaces remotos:**
   `git remote -v`
3. **Publicar la rama por primera vez:**
   `git push -u origin main` *(o `master` según el nombre de la rama principal)*

Nota Técnica: El indicador `-u` (`--set-upstream`) establece la relación de seguimiento entre la rama local y la remota. Una vez configurado, solo se requerirá ejecutar `git push` o `git pull` en las siguientes iteraciones.